
```
(( 'From', 'fairest', 'creatures', 'we'), 1)
```

This n -gram is the beginning of Shakespeare's Sonnet 1. It's fun to look at some of the interesting 4-grams we can randomly find in Shakespeare's works. For example, if you run `print(list(counted_grams)[10])`, you can see that the 10th 4-gram in Shakespeare's works is "rose might never die." If you run `print(list(counted_grams)[240000])`, you can see that the 240,000th n -gram is "I shall command all." The 323,002nd is "far more glorious star" and the 328,004th is "crack my arms asunder." But we want to do phrase completion, not just $n + 1$ -gram browsing. We need to find the subset of $n + 1$ -grams whose first n elements match our search term. We can do that as follows:

```
matching_terms = [element for element in list(counted_grams.items()) if  
                    \element[0][: -1] == tuple(split_term)]
```

This list comprehension iterates over every $n + 1$ -gram and calls each element as it does so. For each element, it checks whether `element[0][: -1] == tuple(split_term)`. The left side of this equality, `element[0][: -1]`, simply takes the first n elements of each $n + 1$ -gram: the `[: -1]` is a handy way to disregard the last element of a list. The right side of the equality, `tuple(split_term)`, is the n -gram we're searching for ("life is a"). So we're checking for $n + 1$ -grams whose first n elements are the same as our n -gram of interest. Whichever terms match are stored in our final output, called `matching_terms`.

Selecting a Phrase Based on Frequency

Our `matching_terms` list has everything we need to finish the job; it consists of $n + 1$ -grams whose first n elements match the search term, and it includes their frequencies in our corpus. As long as there is at least one element in the matching terms list, we can find the element that occurs most frequently in the corpus and suggest it to the user as the completed phrase. The following snippet gets the job done:

```
if(len(matching_terms)>0):  
    frequencies = [item[1] for item in matching_terms]  
    maximum_frequency = np.max(frequencies)  
    highest_frequency_term = [item[0] for item in matching_terms if item[1] ==
```
